

APIs and MCP: A Complete Study Guide

A structured reference covering APIs, HTTP, REST, and the Model Context Protocol (MCP)

1. Fundamentals

1.1 The Problem APIs Solve

Why does it exist? Software systems constantly need to talk to each other — a mobile app needs data from a restaurant's server, a frontend needs data from a backend, an AI assistant needs to read a file. None of these systems can simply reach into each other's internal memory or database. They need a **controlled, agreed-upon way** to ask for things and receive answers.

Intuition Imagine a food-delivery app. It doesn't permanently store every restaurant's menu — it asks the restaurant's server for it, on demand:

```
Your phone — "Give me the menu" —> Restaurant's server
Your phone <— menu data — Restaurant's server
```

That request/response mechanism, and the rules that govern it, is what an API provides.

1.2 What Is an API?

What is it?

Key idea: An **API (Application Programming Interface)** is a defined interface through which one software component can interact with another.

Intuition — the waiter analogy You don't walk into a restaurant kitchen and cook your own food. You tell a waiter what you want; the waiter talks to the kitchen; the kitchen prepares it; the waiter brings it back.

```
YOU → "One biryani" → WAITER → KITCHEN → (prepares) → WAITER → YOU
```

You never need to know how the kitchen is organized, who the chef is, or how ingredients are stored. You only need to know **what you can ask for, and how to ask for it**. That's exactly what an API gives a piece of software: a contract, not a view into internal implementation.

Key idea: An API hides implementation details behind an interface. The caller only needs to know the *contract* (what can be requested and how), not the internal logic.

Example A weather app asks a weather API for "weather in Coimbatore" and gets back:

```
{ "city": "Coimbatore", "temperature": 28, "humidity": 72 }
```

The app has no idea how the weather company actually collects this data — and it doesn't need to.

Common misconception

Common mistake: Thinking an API is always a website or that visiting a URL in a browser is the same as calling an API.

A **website** (<https://example.com>) is designed for humans to read. An **API endpoint** (<https://example.com/api/users>) returns structured data (like JSON) meant for a *program* to consume, not for a human to browse.

	Website	API
Audience	Humans	Programs
Output	HTML pages	Structured data (JSON, XML, etc.)
Purpose	Present information visually	Exchange data/functionality

1.3 Client and Server Roles

Every API interaction has two sides:

- **Client** — the program *asking* for something (a React frontend, a mobile app, a script, another server, an AI application).
- **Server** — the program *providing* the functionality or data.



Diagram 1

Key nuance: The API is not the server itself — it’s the **interface/contract exposed by** the server. A client interacts with the API; behind that API, the server can be built however its developers choose.

A common chain looks like:

```
React App (Client) → HTTP request → Node/Express (Server) → SQL query → PostgreSQL (Database)
```

Common mistake: Assuming every hop in a system uses “an API.” A backend usually talks to its database using a **database driver/protocol**, not an API in the same sense as the client-facing API. Only genuinely interface-exposing hops (e.g., Backend A calling Backend B’s HTTP interface) count as APIs.

Interview Perspective — Section 1

- **Q: What is an API, in one sentence?** A defined interface through which one software component can interact with another, hiding implementation details.
- **Q: Is a website an API?** No — websites serve HTML for humans; APIs serve structured data for programs.

2. HTTP Fundamentals

2.1 What Is HTTP?

What is it? HTTP (**HyperText Transfer Protocol**) is one of the main protocols used for communication between clients and servers on the web. Think of it as the *language and rules* of the conversation — how requests and responses must be structured.

Key idea: API and HTTP are **not the same thing**.

- **API** = the interface/contract.
- **HTTP** = one common *protocol* used to implement/communicate through that interface.

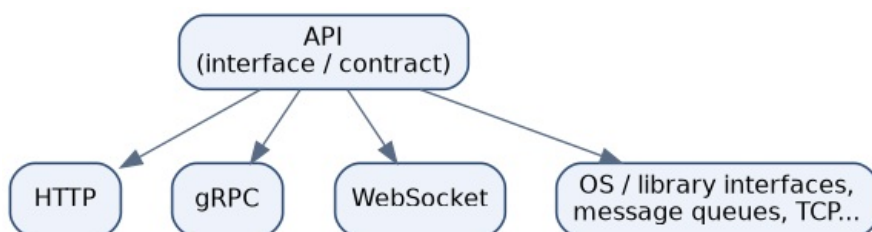


Diagram 2

Common misconception

Common mistake: Believing “API means HTTP.” An API can be exposed over HTTP, gRPC, WebSockets, or even as a plain programming-library interface with no network involved at all.

2.2 HTTP Methods

HTTP standardizes the *kind of operation* being requested. This maps naturally onto CRUD (Create, Read, Update, Delete):

Method	Meaning	CRUD	Example
GET	Retrieve something	Read	GET /users
POST	Create something	Create	POST /users
PUT	Replace a resource	Update (full)	PUT /users/5
PATCH	Partially modify a resource	Update (partial)	PATCH /users/5
DELETE	Delete something	Delete	DELETE /users/5

2.3 Endpoints, Routes, and Resources — Precise Definitions

This is a place where casual explanations often blur terms. Here are the precise definitions, refined from an initial looser explanation given earlier in the learning process:

Endpoint — a network-accessible interface through which a client can perform a *particular operation* on a server. For an HTTP API, an endpoint is identified by **HTTP method + URL/path** together — not the path alone.

Route — the URL *pattern* that maps an incoming request to server-side handling logic (e.g., `/users/{id}`).

Resource — the conceptual “thing” being represented and acted upon (e.g., “user 42”).

Worked example — resolving the “are `/users/42` and `/users/53` different endpoints?” question

```
Route pattern:    /users/{id}
Concrete requests: GET /users/42
                  GET /users/53
                  GET /users/100
```

These are different *concrete invocations* of the *same route pattern*, each operating on a different resource instance. Meanwhile:

```
GET /users    → different endpoint from →    POST /users
```

Same path, different HTTP method → different operations/endpoints, because the endpoint is defined by **method + path**, not path alone.

Common mistake: Saying “the endpoint is `/users`.” More precisely: `/users` is a path/route; `GET /users` and `POST /users` are two distinct endpoints sharing that path.

Hierarchy to remember:

```
API
├── HTTP API
│   ├── GET /users/{id}    ← endpoint (method + route)
│   │   ├── GET          ← HTTP method
│   │   └── /users/{id}  ← route pattern
│   │       └── 42        ← path parameter → identifies a resource instance
```

2.4 HTTP Status Codes

The server communicates outcome via status codes.

Code	Class	Meaning
200 OK	2xx Success	Request succeeded
201 Created	2xx Success	A resource was successfully created
400 Bad Request	4xx Client error	The request was invalid
401 Unauthorized	4xx Client error	You need to authenticate
403 Forbidden	4xx Client error	You're authenticated, but not allowed to do this
404 Not Found	4xx Client error	The requested resource doesn't exist
500 Internal Server Error	5xx Server error	Something went wrong on the server

Mental model: 2xx → success, 3xx → redirection, 4xx → the client's fault, 5xx → the server's fault.

Common mistake: Confusing 401 and 403. 401 means "I don't know who you are — please authenticate." 403 means "I know who you are — you're just not allowed."

2.5 JSON

JSON (JavaScript Object Notation) is a standardized, language-independent data format:

```
{
  "name": "John",
  "age": 21,
  "skills": ["Python", "React", "Docker"]
}
```

Python, JavaScript, Java, C#, and Go can all parse it — which is exactly why APIs commonly use it as their data representation.

2.6 Anatomy of a Request

Consider a full request:

```
POST /users?notify=true HTTP/1.1
Content-Type: application/json
Authorization: Bearer xyz

{
  "name": "Alice"
}
```

Part	Value	Purpose
Method	POST	The operation being requested
Path	/users	Identifies the resource/collection
Query parameter	?notify=true	Extra, optional request-level options
Header: Content-Type	application/json	Tells the server how to interpret the body
Header: Authorization	Bearer xyz	Identity/credential information
Body	{"name": "Alice"}	The actual data payload

Why isn't Authorization just put inside the JSON body? Headers carry *metadata about the request* (who's asking, what format the data is in) separately from the *actual payload*. This lets infrastructure — proxies, gateways, logging systems, servers — inspect and act on identity/format information without needing to parse or understand the body at all. It also lets the same authorization mechanism work uniformly across GET requests (which typically have no body).

Interview Perspective — Section 2

- **Q: Is `/users/42` a different endpoint than `/users/53`?** They're different *concrete invocations* of the same route pattern `/users/{id}`, not different endpoint *definitions*.
- **Q: What's the difference between a header and the body?** Headers = metadata about the request/response; body = the actual data payload.
- **Q: What do 401 and 403 each mean?** 401 = not authenticated; 403 = authenticated but not authorized.

3. REST APIs

3.1 What REST Actually Is

What is it?

Key idea: REST (Representational State Transfer) is an **architectural style** for designing networked applications — it is *not* a protocol like HTTP.

A RESTful API typically:

- Models things as **resources**
- Identifies resources using **URLs**
- Uses **standard HTTP methods** to operate on them
- Is **stateless**
- Transfers **representations** (commonly JSON) of resource state

Example — REST style vs RPC style

REST models the URL as a *resource* and the method as the *operation*:

```
GET    /users/42    → retrieve user 42
PUT    /users/42    → replace user 42
DELETE /users/42    → delete user 42
```

An RPC-style HTTP API instead encodes the *operation* itself into the URL:

```
POST /getUser    {"id": 42}
POST /deleteUser {"id": 42}
```

Neither is inherently “wrong” — but only the first is REST-style, because REST’s core idea is:

Remember: The URL identifies *what* you’re acting on (the resource); the HTTP method describes *what you want to do with it*. Putting an action verb into the URL (`/getUser` , `/deleteUser`) is an RPC pattern, not REST.

3.2 HTTP API vs REST API — Don’t Confuse Them

These are answers to two *different* questions:

	Describes	Question it answers
HTTP API	Communication mechanism	“How does this API communicate?” → “Over HTTP.”
REST API	Architectural design	“How is this API designed?” → “Resource-oriented, stateless, standard-HTTP-method semantics.”

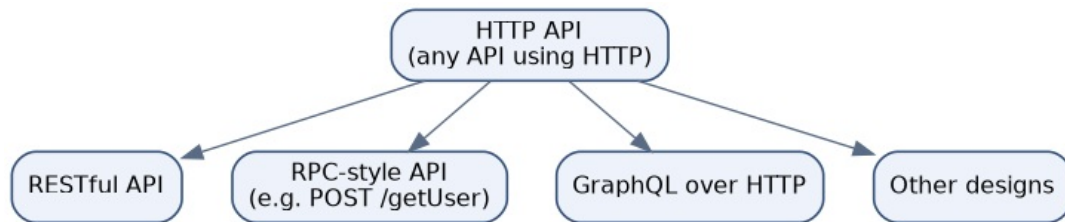


Diagram 3

Key idea: Every REST API commonly uses HTTP, but **not every HTTP API is RESTful**.

3.3 Statelessness

What is it?

Each request from a client to a server must contain **all information the server needs to process it**; the server does not rely on remembered state from *previous requests in that client's session*.

Common misconception

Common mistake: Believing statelessness means “the server can’t store any data.” It absolutely can — users, orders, products, and full databases are all stored server-side. What statelessness actually restricts is **client/server interaction state**: the server shouldn’t need to recall “what this specific client asked earlier” to make sense of the current request.

Example — how stateless APIs still support logins A user logs in once; the server doesn’t need to “remember the conversation” afterward. Instead, each subsequent request carries its own proof of identity, typically via a header:

```
Authorization: Bearer <token>
```

The server looks up whatever it needs (e.g., the user’s account row) fresh, on every request, using information *in that request* — it isn’t relying on memory of prior requests.

3.4 PUT vs PATCH

Common mistake: (“PUT creates a new data point, PATCH updates existing data”) — this is a common misunderstanding. The **real** distinction is about *replace vs. partial modify*, not *create vs. update*.

	PUT	PATCH
Meaning	Replace the resource’s full representation at a known URI (can create it if the API allows)	Partially modify part of the resource
Example	<code>PUT /users/42 {"name": "John", "age": 25}</code> (must send the <i>whole</i> representation)	<code>PATCH /users/42 {"age": 26}</code> (send only the changed field)
Idempotent?	Generally yes	Not inherently idempotent

3.5 Idempotency

What is it?

An operation is **idempotent** if sending the same request multiple times has the **same intended effect** on server state as sending it once.

Why does it matter? Networks are unreliable — clients sometimes retry requests without knowing whether the first one succeeded. Idempotent operations are safe to retry blindly; non-idempotent ones are not.

Example

```
PUT /users/42 {"name": "Alice"}
```

Sent once → user 42’s name is “Alice”. Sent 10 times → user 42’s name is *still* “Alice.” **Idempotent**.

```
POST /users {"name": "Alice"}
```

Sent once → creates user 101. Sent again → could create user 102, then 103... **Not idempotent** — each call can create a new resource.

Common mistake: Believing “idempotent” means “the request can’t be repeated” or “PUT never changes state.” Idempotency is about the *end state being the same*, not about *nothing happening*.

3.6 Why GET Shouldn’t Have Side Effects

GET /deleteUser/42 is bad API design **even if it technically works**, because HTTP methods carry semantic guarantees: GET is meant to be **safe** (no intended state change) and used purely for retrieval. Browsers, caches, proxies, crawlers, and other tooling all assume GET requests are side-effect-free and may prefetch, cache, or repeat them freely — which would be catastrophic if GET could silently delete data. The correct design uses the method whose semantics match the intent:

```
DELETE /users/42
```

3.7 Authentication vs Authorization

	Question it answers	Example
Authentication	Who are you?	Logging in
Authorization	What are you allowed to do?	“Alice can read users but not delete them”

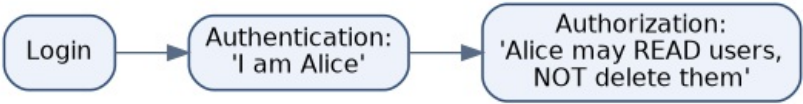


Diagram 4

Interview Perspective — Section 3

- **Q: Is REST a protocol?** No — it’s an architectural style. HTTP is the protocol REST commonly runs over.
- **Q: PUT vs PATCH?** PUT replaces the full resource representation; PATCH partially modifies it.
- **Q: Define idempotency and give a non-idempotent example.** Same request repeated → same intended end state. POST is a classic non-idempotent example since repeats can create multiple resources.
- **Q: How can a REST API be stateless if users log in and have accounts stored?** Statelessness is about *not relying on remembered request history*; stored account data (in a database) is unrelated — each request still must carry its own credentials/context.

4. Other Ways APIs Communicate (Beyond HTTP)

Since **API ≠ HTTP**, it’s worth knowing the alternatives:

Protocol	Typical Use Case
HTTP/HTTPS	Most common for web-facing APIs (REST, general HTTP APIs); often paired with JSON
gRPC	Popular for microservices / internal service-to-service calls; defines services like <code>getUser(UserRequest)</code> instead of URL routes
WebSockets	Persistent, two-way connections — chat apps, live notifications, multiplayer games, live dashboards. Either side can push messages, unlike the request→response HTTP model
TCP / Unix sockets / message queues	Lower-level mechanisms an API can be built directly on top of

Remember: Memorize `API = interface/contract`, and `HTTP = one common protocol used to communicate through that interface` — never `API = HTTP`.

5. MCP Fundamentals

5.1 The Problem MCP Solves

Why does it exist? Imagine an AI assistant that needs to check GitHub issues, query a MySQL database, read local files, and check the weather. Without a standard, every single one of these needs **custom, service-specific integration logic**:

```
AI
├─ custom weather integration
├─ custom GitHub integration
├─ custom filesystem integration
├─ custom database integration
└─ custom Slack integration ...
```

That doesn't scale — every new tool means writing new bespoke integration code, and every AI application repeats this same work.

MCP (Model Context Protocol) was created to provide a **standardized way** for AI applications to discover and interact with external tools, data, and capabilities.

5.2 What MCP Actually Is

What is it?

Key idea: MCP is a **protocol** — a set of rules that lets AI applications and external capability providers communicate in a standardized way, the same role HTTP plays for general web communication.

```
HTTP → standardized communication between web clients and servers
MCP  → standardized communication between AI applications and external tools/data/context
```

Common misconception

Common mistake: “MCP is like an API for AI” is a fine *starting* intuition but is imprecise. More precisely: **an API** is usually the interface a *particular service* exposes (e.g., GitHub's API). **MCP** is a standardized, AI-facing protocol that can sit **between** an AI application and many such services — including services that already have their own APIs.

5.3 MCP vs API — The Relationship

MCP and APIs are **not competitors** — they typically work together. A very common real shape is:



Diagram 5

	API	MCP
Answers the question	"How can one software system communicate with another?"	"How can an AI application discover and use external capabilities in a standardized way?"
Typical example	React → Backend API , App → Weather API	AI → MCP → tools/resources → external systems
Relationship	MCP servers often call APIs internally	MCP can sit <i>on top of</i> existing APIs

Key idea: An **API** provides an interface for software components to interact. **MCP** provides a standardized protocol specifically for **AI applications** to discover and interact with external capabilities such as tools and resources — often by wrapping existing APIs.

5.4 MCP Architecture

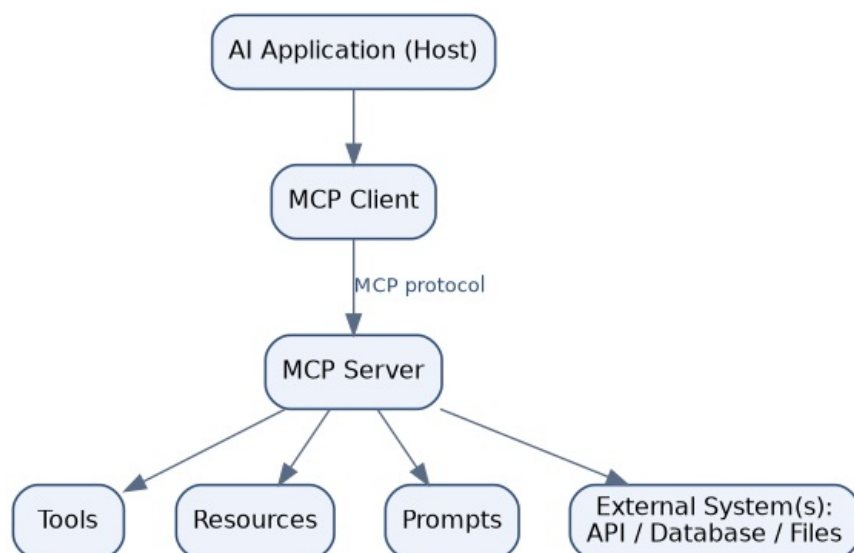


Diagram 6

Component	Role
Host / AI Application	The application that wants to use external capabilities (e.g., an AI coding assistant)
MCP Client	The component inside the host that maintains the standardized MCP communication channel
MCP Server	Implements and exposes tools/resources/prompts through MCP; contains the <i>actual</i> integration logic
External System	GitHub, a database, the filesystem, another API — whatever the server ultimately talks to

Common mistake: Thinking the AI *directly* touches GitHub, MySQL, etc. In reality, the AI talks to the MCP Client → MCP Server, and the **server** is the one that actually talks to the external system.

5.5 Tools, Resources, and Prompts

Concept	What it represents	Examples
Tools	Things the AI can <i>do</i>	<code>create_issue()</code> , <code>send_email()</code> , <code>search_database()</code> , <code>delete_file()</code>
Resources	Things the AI can <i>read/access</i> as context	File contents, database info, documentation
Prompts	Reusable prompt templates/workflows an MCP server can expose	A pre-built “summarize this repo” workflow

Interview Perspective — Section 5

- **Q: Is MCP an API?** No — MCP is a protocol. It standardizes how an AI (via an MCP client) discovers and invokes capabilities exposed by an MCP server; the server may itself call ordinary APIs underneath.
- **Q: What problem does MCP solve that plain APIs don’t?** It eliminates the need for one bespoke integration per external tool by giving every AI application a single, standard way to discover and call capabilities.
- **Q: Name MCP’s three core capability types.** Tools, Resources, Prompts.

6. MCP Deep Dive

6.1 MCP Protocol vs MCP Server — The USB Analogy

Why does it exist? (clarifying a common early confusion) A natural early question is: “Are MCP servers built for specific tools like GitHub or MySQL, or can one server work for everything?”

Key idea: MCP is a general-purpose protocol. An **MCP server** is a *specific implementation* that exposes a particular set of capabilities through that protocol. The protocol doesn’t change per tool — only what a given server chooses to expose does.

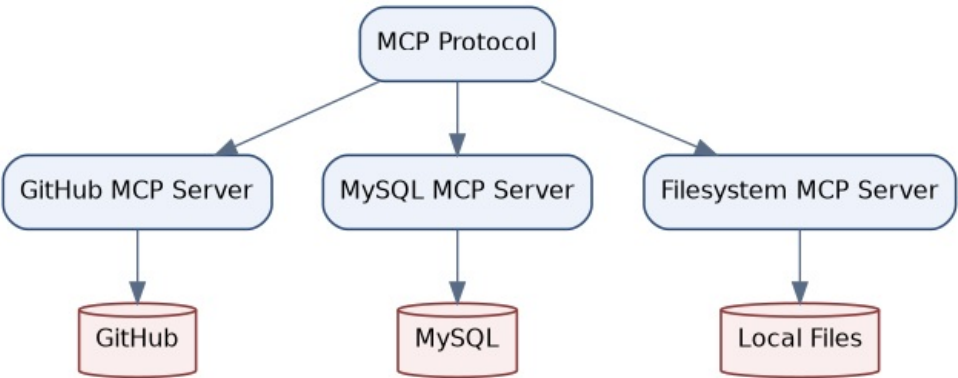


Diagram 7

Analogy

Think of MCP like **USB**: USB is a standardized connection protocol. A keyboard, mouse, storage drive, and camera are all wildly different devices, but they all speak the same USB protocol. Similarly, a GitHub MCP server, a MySQL MCP server, and a filesystem MCP server all speak the same MCP protocol while exposing completely different capabilities.

Could one giant MCP server expose everything (GitHub + MySQL + files + email)? Technically yes — but it’s usually undesirable. Splitting servers by capability domain lets each one have independent:

- Tools
- Authentication/credentials
- Permissions
- Deployment lifecycle
- Security boundaries

Interview-quality definition: An MCP server is a server-side implementation of the Model Context Protocol that exposes tools, resources, and/or prompts to an MCP client. The protocol is general-purpose; the capabilities exposed by any particular server depend entirely on its implementation and the systems it integrates with.

6.2 MCP Security — The Most Important Section

This is the area with the deepest misconceptions worth correcting carefully.

The core question: *If someone builds a general-purpose MySQL MCP server, how do you stop the AI from accessing sensitive data (e.g., account balances, password hashes) in a specific application (like a bank)?*

Key idea (the golden rule): Never treat the AI as a trusted security boundary. The AI deciding “I think the user wants the account balance” must never be sufficient permission to retrieve it. Security must be enforced by layers **underneath** the AI, not by hoping the AI behaves.

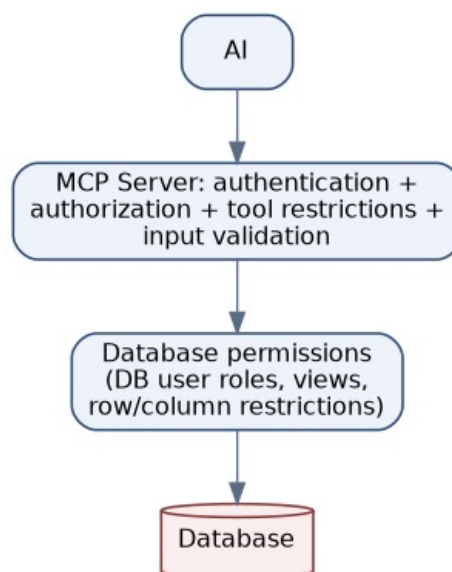


Diagram 8

There are several *independent, stackable* layers of defense — this is defense in depth:

1. **Restrict the database account itself.** Even if the MCP server is compromised or the AI misbehaves, a database user (e.g., `ai_readonly`) that can only `SELECT` from `customers` and `transactions` — but not `credentials` — cannot leak passwords, because MySQL itself will reject the attempt.
2. **Don’t expose raw query access; expose narrow, purpose-built tools.** Instead of a dangerous `execute_sql(query)` tool, expose specific tools like `get_customer_name(customer_id)` or `get_order_status(order_id)`. This removes the AI’s ability to construct arbitrary queries altogether.
3. **Restrict which tools are exposed at all**, even if the server *could* support more:

MCP Server	
read_customer	✓ exposed
read_transactions	✗ disabled
delete_customer	✗ disabled
transfer_money	✗ disabled

Minimizing exposed capability surface is itself a security control, independent of whether each tool is individually “safe.”

4. **Enforce authorization *inside* each tool**, not just at the tool-exposure level. Even `read_customer(user_id)` should check “is this caller allowed to access `user_id`?” before returning data — having the *tool exist* is not the same as the *caller being allowed to use it for any input*.
5. **Enforce authorization at the account MCP authenticates as.** If the MCP server connects to Jira using a particular integration account/token, that account’s own permissions form another hard boundary — the MCP server cannot magically exceed what its underlying credentials allow.

Common mistake: (“The request is not accepted in the MySQL DB”) — this is only *one* possible layer, and it’s not something MCP does automatically. MCP itself has no opinion on whether a query should succeed; the *database* (or the MCP server’s own validation logic) is what actually rejects it. **MCP does not automatically make tool usage secure.**

6.3 Standardization vs. Customization — Resolving “Doesn’t Custom Config Defeat the Purpose?”

A very natural objection: “If I have to configure/restrict an MCP server per deployment, doesn’t that defeat the purpose of a standard protocol — isn’t that just custom integration again?”

Resolution:

Key idea: MCP standardizes the **communication/interaction model** between an AI and a capability provider. It does **not**, and was never meant to, standardize an application’s business authorization rules — those are inherently application-specific, exactly the way HTTP being a standard protocol doesn’t mean every website has the same permissions.

The same MCP *server implementation* can be deployed multiple times with different credentials, database roles, and exposed-tool configurations for different customers/deployments — the protocol stays identical; only the configuration changes:

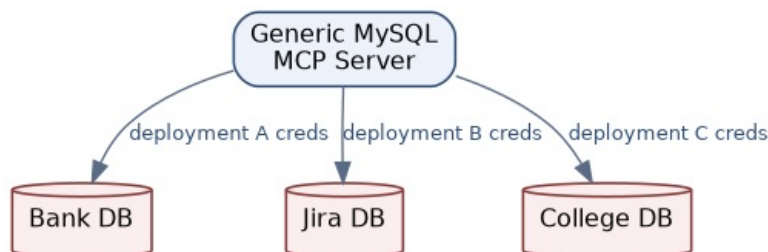


Diagram 9

You do **not** need a `Bank-MCP-Server`, `Jira-MCP-Server`, and `College-MCP-Server` as separate *protocols* — you need one protocol-compliant server, configured differently per deployment.

What MCP standardizes	What stays application-specific
Client ↔ server communication format	What tools a server provides
Capability discovery mechanism	Business logic
Tool invocation mechanism	Database schema
Results/error representation	User permissions
Resources/prompts interaction	External-service credentials
Protocol-level lifecycle/negotiation	Internal tool implementation

Remember: MCP = communication/protocol standard. **Security = authentication + authorization + permissions**, implemented by the host/server/integrated system — MCP participates in enabling secure communication but does not replace application-level authorization.

Interview Perspective — Section 6

- **Q: Does MCP automatically prevent an AI from running a dangerous SQL query?** No. MCP has no built-in awareness of query safety. Protection must come from restricted database credentials, narrowly scoped tools, input validation in the MCP server, and/or database-level permission checks.
- **Q: If every company has different authorization rules, how can MCP still be “standardized”?** Because MCP standardizes the *protocol/interaction mechanism* (discovery, invocation, results format), not the *authorization policy* — exactly as HTTP is standardized while every website’s permission model differs.
- **Q: Why might you split GitHub, MySQL, and filesystem capabilities into separate MCP servers instead**

of one big server? Independent credentials, permissions, deployment lifecycles, and security boundaries — isolating a compromise or misconfiguration in one domain from affecting the others.

7. Putting It All Together

7.1 The Full Mental Model

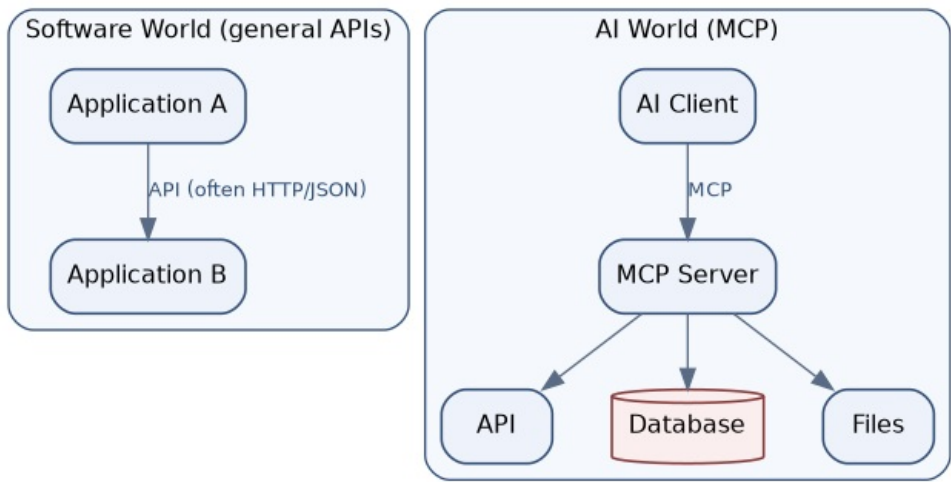


Diagram 10

7.2 A Concrete End-to-End Example

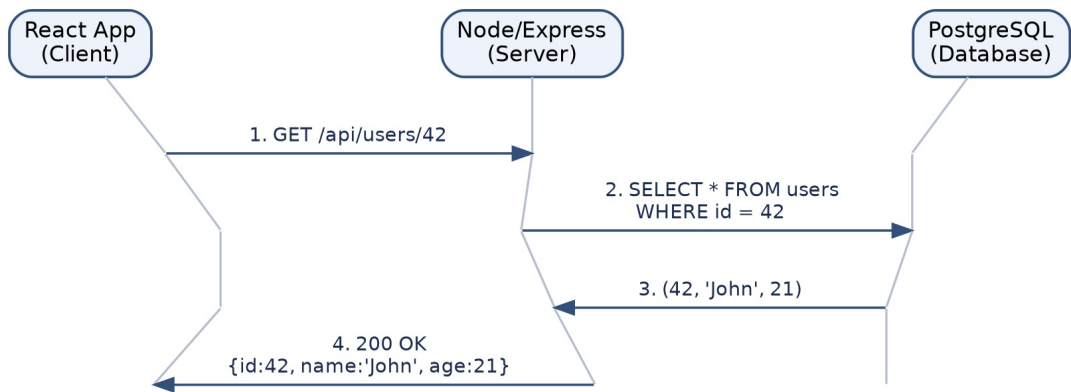


Diagram 11

And the MCP-flavored equivalent for an AI accessing GitHub:

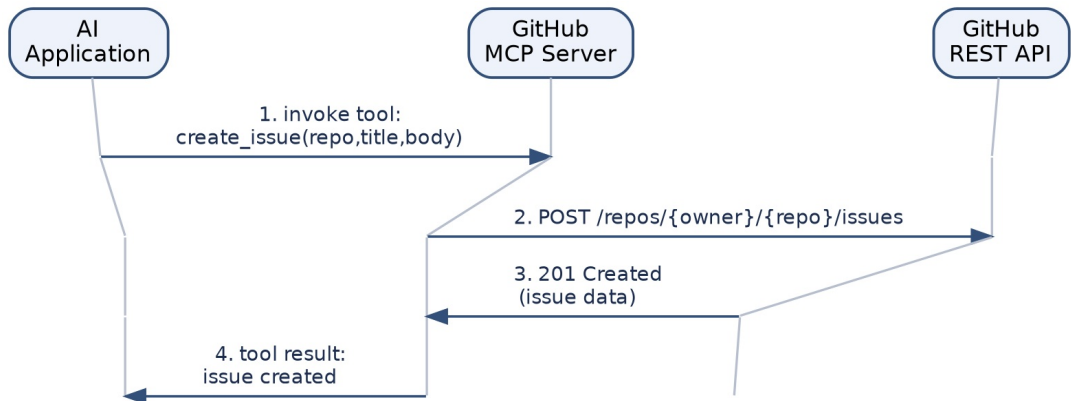


Diagram 12

7.3 Definitions to Lock In

Term	Precise Definition
API	A defined interface through which one software component can interact with another.
HTTP API	An API whose interface is accessed using the HTTP protocol.
REST	An architectural style defining constraints for designing networked systems around resources and their representations.
REST API	An API designed according to REST’s constraints, commonly (but not necessarily) using HTTP.
Endpoint	A network-accessible interface through which a specific API operation can be invoked; for HTTP APIs, identified by HTTP method + URL/path.
Route	The URL pattern mapping a request to server-side handling logic.
MCP	A protocol that standardizes how AI applications discover and interact with external tools, resources, and prompts.
MCP Server	A server-side implementation of MCP exposing tools/resources/prompts to an MCP client, wrapping the actual integration logic.

8. Common Mistakes Recap

These are mistakes made and corrected during this learning journey — worth reviewing directly, since they represent the exact edges where intuition and precision diverge.

Mistake	Why it’s wrong	Correct version
“The endpoint is <code>/users</code> .”	Ignores that the HTTP method is part of what defines an operation	Endpoint = method + path, e.g. <code>GET /users/{id}</code>
“PUT creates a new data point, PATCH updates existing data.”	Confuses create-vs-update with replace-vs-partial-modify	PUT replaces a resource’s full representation (may create if allowed); PATCH partially modifies it
“Idempotent means the request can’t be repeated.”	Idempotency is about <i>effect</i> , not repeatability	Idempotent = repeating the request produces the same intended end state
“Backend always talks to the database using an API.”	Overgeneralizes “API” to any inter-component communication	Backend-to-database communication typically uses a database driver/protocol, not an API
“MCP is an interface between an LLM and tools.”	Slightly imprecise — undersells that it’s a protocol, and that servers may call ordinary APIs underneath	MCP standardizes communication between an MCP host/client and MCP servers, which then expose tools/resources/prompts — often by calling normal APIs internally
“Restricting which tools an MCP server exposes per deployment defeats MCP’s purpose.”	Conflates protocol standardization with authorization policy	MCP standardizes the <i>how</i> of communication; <i>what</i> is authorized is inherently application-specific, just like with any standard protocol (e.g., HTTP)
“The database rejecting a bad query means MCP is secure.”	Attributes security entirely to one layer, and to MCP itself	Security is layered: restricted DB credentials, narrow tool design, in-tool authorization checks, and DB permissions — MCP itself enforces none of this automatically

Don't Confuse These (Quick Interview Reference)

These are the pairs/groups of terms interviewers most often use to probe whether your understanding is precise or just surface-level. Each one is a common trap.

API vs HTTP

	API	HTTP
What it is	An interface/contract for one software component to interact with another	A communication <i>protocol</i> — one specific way to move requests/responses over a network
Scope	Broader concept	Narrower — one implementation choice
Alternatives	Can be implemented over HTTP, gRPC, WebSockets, TCP, or even a local library call	N/A — HTTP is itself one of the options

One-liner: API is *what* two systems agree to communicate about; HTTP is *how* the bytes travel. Every REST API uses HTTP, but not every API does.

REST vs HTTP

	REST	HTTP
What it is	An architectural style — a set of design constraints (resource-oriented URLs, standard method semantics, statelessness, representations)	A transfer protocol defining how requests/responses are structured
Category	Design philosophy	Communication mechanism
Relationship	REST APIs are commonly <i>built on top of</i> HTTP	HTTP can carry REST-style APIs, RPC-style APIs, GraphQL, or anything else

One-liner: HTTP is the road; REST is a style of driving on it (obeying resource/method conventions). You can drive on that same road in a non-REST (RPC) style too.

API vs MCP

	API	MCP
What it is	An interface a <i>service</i> exposes for software to interact with it	A standardized protocol for <i>AI applications</i> to discover and invoke external tools/resources/prompts
Who typically uses it	Any software component (frontend, backend, script, AI, etc.)	Specifically AI applications/hosts
Relationship	Can exist completely independently of AI	Often sits <i>on top of</i> existing APIs — an MCP server frequently calls a normal API (e.g., GitHub's REST API) internally

One-liner: Every MCP server usually *uses* an API underneath; MCP is the standardized AI-facing layer, the API is what actually gets called to do the work.

MCP vs Function Calling

This one trips people up because the two are used together constantly, but they operate at different layers.

	Function Calling (Tool Calling)	MCP
What it is	A model capability : the LLM outputs a structured request (“call function X with these arguments”) instead of plain text	A protocol standardizing how tools/resources are <i>described, discovered, and invoked</i> between a host application and external servers
Layer	Inside the model’s output format	Between the host application and external capability providers
Can exist without the other?	Yes — an app can wire an LLM’s function-calling output directly to its own custom backend with zero MCP involved	Yes — MCP defines discovery/invoke mechanics regardless of <i>how</i> the host decided to call a tool
How they combine	The model <i>decides</i> to call a tool and <i>formats</i> the call (function calling)	The host application then routes that call to an MCP server, which executes it and returns a result (MCP)

One-liner: Function calling is the model’s *voice* for requesting a tool call; MCP is the *plumbing* that standardizes how that request reaches, and gets executed by, an external server. You can have function calling with no MCP (custom backend), and conceptually a system can expose MCP tools even if the calling model uses a different internal mechanism to select them.

MCP Host vs MCP Client vs MCP Server

Another classic confusion — these three sound similar but play distinct roles.

	Host	Client	Server
What it is	The AI application itself (e.g., an AI coding assistant, a chat app)	A component inside the host that manages one connection to one MCP server	An external program that exposes tools/resources/prompts via MCP
Cardinality	One host	A host creates one client per server it connects to	Many servers can exist independently; each is capability-specific (GitHub, MySQL, filesystem, ...)
Owns implementation logic?	No — it orchestrates and uses results	No — it just relays protocol messages	Yes — the server contains the actual integration code

AI Application (Host)
├─ MCP Client 1 ─MCP─ GitHub MCP Server
├─ MCP Client 2 ─MCP─ MySQL MCP Server
└─ MCP Client 3 ─MCP─ Filesystem MCP Server

One-liner: One host can maintain several clients simultaneously — one per server — the same way one browser (host) can have several open tabs (clients), each talking to a different website (server).

Tool vs Resource vs Prompt

	Tool	Resource	Prompt
What it represents	An action the AI can invoke (does something, may have side effects)	Read-only context/data the AI can access	A reusable template/workflow , typically surfaced for the <i>user</i> to select
Example	<code>create_issue()</code> , <code>send_email()</code> , <code>delete_file()</code>	File contents, a database record, a documentation page	A pre-built “review this PR” or “summarize this repo” workflow
Who typically triggers it	The model, autonomously, as part of reasoning	The model, to gather context before/while reasoning	Usually the user , explicitly (e.g., a slash command), not the model deciding on its own
Side effects?	Can have them (e.g., writing data)	No — purely informational	Depends on what the prompt’s underlying workflow does once invoked

Common mistake: Assuming everything an MCP server exposes is a “tool.” A read-only file isn’t a tool call target — it’s a resource. A canned, user-selectable workflow isn’t a tool either — it’s a prompt.

MCP vs Agent

	MCP	Agent
What it is	A protocol/connectivity layer for discovering and invoking external tools/resources	A broader system (usually LLM-based) that autonomously plans, reasons over multiple steps, and takes actions toward a goal
Provides autonomy/planning?	No — it’s just plumbing for capability access	Yes — this is the defining feature of an agent
Dependency	An agent <i>can</i> use MCP servers as its toolset	MCP has no concept of “agent” built in — it doesn’t loop, plan, or maintain goals itself

One-liner: MCP gives an AI system *hands* (standardized access to tools/data); an agent is the *decision-making loop* that decides which hand to use, when, and why, often across many steps. You can build an agent without MCP (custom tool-calling), and you can use MCP without building anything that qualifies as an “agent” (e.g., a single-turn assistant that calls one tool and stops).

Interview Questions

Basic

- 1. What is an API?** A defined interface through which one software component can interact with another. Example: `Frontend → API → Backend → Database` — the frontend only needs to know the API contract, not the backend's internals.
- 2. Is an API the same as HTTP?** No. API = interface; HTTP = communication protocol. HTTP is one common way to expose/access an API; others include gRPC, WebSockets, or plain library interfaces.
- 3. What is REST?** An architectural style for designing networked systems. A RESTful API models things as resources, uses resource-oriented URLs, standard HTTP methods, is stateless, and transfers representations like JSON.
- 4. What's the difference between authentication and authorization?** Authentication = who are you (e.g., logging in). Authorization = what are you allowed to do (e.g., "Alice can read users but not delete them").
- 5. What is JSON and why do APIs use it?** A language-independent, standardized text format for representing structured data; virtually every programming language can parse it, making it a convenient common format between client and server.

Intermediate

- 6. PUT vs PATCH — what's the difference?** PUT generally replaces a resource's full representation (and may create it if the API allows). PATCH partially modifies a resource. Example: `PUT /users/42 {"name":"John","age":25}` vs `PATCH /users/42 {"age":26}`. PUT is generally idempotent; PATCH is not inherently idempotent.
- 7. What is idempotency? Give an idempotent and a non-idempotent example.** An operation is idempotent if repeating the same request has the same intended effect as doing it once. `PUT /users/42 {"name":"John"}` repeated 5 times still leaves user 42 named "John" (idempotent). `POST /users {"name":"John"}` repeated 5 times could create 5 different users (non-idempotent).
- 8. Why shouldn't `GET /deleteUser/42` be used to delete a resource?** `GET` is expected to be safe (no intended side effects); browsers, caches, and proxies assume this and may freely repeat or prefetch `GET` requests. Deletion should use `DELETE /users/42` so the operation's semantics are explicit and correctly handled by tooling.
- 9. How can a REST API be stateless if the server stores user account data?** Statelessness is about not relying on remembered *client interaction history* between requests — not about whether the server stores data at all. The server can store users, orders, etc.; each request just needs to independently carry whatever context (e.g., an auth token) it needs to be understood.
- 10. What is MCP, and why not just let an AI call HTTP APIs directly?** MCP (Model Context Protocol) is a protocol standardizing how AI applications discover and invoke external tools, resources, and prompts. Without it, an AI needs bespoke integration logic (auth, endpoints, parameters, error handling) for every single external service. MCP moves that per-service complexity into MCP servers, giving the AI one consistent interaction mechanism across all of them.

Advanced

- 11. Is MCP itself an API? Explain the relationship between API, HTTP API, MCP, and MCP Server.** MCP is not an API in the traditional sense — it's a protocol for AI-to-capability interaction. An MCP **server** exposes capabilities (tools/resources/prompts) via the MCP protocol, and that server often calls a normal API (e.g., GitHub's REST API) underneath to actually fulfill requests. So: `AI → (MCP protocol) → MCP Server → (HTTP API) → External Service`.
- 12. Why can a GitHub MCP server, a MySQL MCP server, and a filesystem MCP server all "speak MCP" despite exposing totally different capabilities?** Because MCP is a general-purpose protocol (like USB), not a tool-specific one. Any MCP server implementation, regardless of what it wraps, communicates using the same discovery/invoke/result mechanisms; the *capabilities* it exposes are up to its implementer.
- 13. An AI with access to a MySQL MCP server asks it to run `SELECT password_hash FROM users;`. Does MCP prevent this automatically? Where should the restriction actually live?** No, MCP does not automatically

prevent this — it has no inherent concept of “safe” queries. Restriction must come from one or more of: a restricted database user without access to that column/table, narrowly scoped tools instead of raw `execute_sql`, in-tool authorization checks, and/or MCP-server-level input validation. Defense should be layered, not reliant on the AI “choosing” not to ask.

14. If every company has different authorization rules, database schemas, and permissions, how can MCP still be considered a standardized protocol? Because standardization applies to the *interaction mechanism* — how capabilities are discovered, invoked, and how results/errors are communicated — not to *what* any given deployment is authorized to do. This is the same relationship HTTP has to differing website permission models: the protocol is uniform; the policy is not.

15. Design question: An AI Host connects via an MCP Client to both a GitHub MCP Server and a MySQL MCP Server, which respectively call the GitHub API and MySQL. Which parts are standardized by MCP, and which are application-specific? Standardized: client/server communication format, capability discovery, tool description/invocation mechanism, results/error representation, protocol-level lifecycle negotiation. Application-specific: which tools exist on each server, which queries/operations are permitted, which credentials are used, which accounts are accessed, business authorization rules, and each tool’s internal implementation.

Final Cheat Sheet

Core definitions

- **API** — interface/contract for software components to interact.
- **HTTP** — a communication protocol; one of several ways to implement an API (others: gRPC, WebSockets, TCP-level protocols).
- **HTTP API** — an API accessed via HTTP; not necessarily REST.
- **REST** — an architectural style (resource-oriented, stateless, standard HTTP method semantics); not a protocol.
- **Endpoint** — HTTP method + path, e.g. `GET /users/{id}` — the unit of a specific operation.
- **Route** — the URL pattern that maps to server logic.
- **Idempotent** — repeating a request yields the same intended end state (PUT: yes; POST: generally no).
- **Stateless (REST)** — each request is self-contained; server doesn't rely on remembered interaction history (but can still store data).
- **MCP** — protocol standardizing AI ↔ external-capability discovery and invocation.
- **MCP Server** — implementation exposing tools/resources/prompts via MCP, usually wrapping real APIs/DBs/files underneath.

HTTP methods ↔ CRUD

GET	→ READ
POST	→ CREATE
PUT	→ REPLACE
PATCH	→ PARTIAL UPDATE
DELETE	→ DELETE

Status code families

2xx → Success	4xx → Client's fault
3xx → Redirection	5xx → Server's fault

Authentication vs Authorization

Authentication → "Who are you?"
Authorization → "What are you allowed to do?"

MCP architecture

AI Host → MCP Client → (MCP protocol) → MCP Server → Tools/Resources/Prompts → External System (API/DB/Files)

MCP security golden rule

Never treat the AI as a trusted security boundary. Enforce restrictions via: restricted credentials, narrowly scoped tools (not raw query execution), in-tool authorization checks, and underlying system (DB/API) permissions — stacked as defense in depth.

MCP standardizes vs. doesn't standardize

Standardizes:	Does NOT standardize:
client/server communication	what tools a server exposes
capability discovery	business logic / DB schema
tool invocation mechanism	user permissions
results/error format	external-service credentials
lifecycle/capability negotiation	internal tool implementation

Glossary

- **API (Application Programming Interface)** — A defined interface through which one software component can interact with another.
- **Authentication** — Verifying *who* a caller is.
- **Authorization** — Determining *what* an authenticated caller is allowed to do.
- **Body (request/response body)** — The actual data payload sent in an HTTP message.
- **Client** — The program initiating a request (frontend, script, AI application, etc.).
- **CRUD** — Create, Read, Update, Delete — the four basic data operations, commonly mapped to POST/GET/PUT-PATCH/DELETE.
- **Defense in depth** — Layering multiple independent security controls so that failure of one layer doesn't fully compromise the system.
- **Endpoint** — A specific API operation, identified (for HTTP APIs) by HTTP method + URL/path.
- **gRPC** — A remote procedure call framework commonly used for service-to-service communication, an alternative to REST/HTTP-style APIs.
- **Header (HTTP header)** — Metadata attached to an HTTP request/response (e.g., `Content-Type`, `Authorization`), separate from the body.
- **HTTP (HyperText Transfer Protocol)** — A protocol defining how clients and servers structure requests/responses on the web.
- **HTTP API** — An API whose communication happens over HTTP; may or may not be RESTful.
- **Idempotency** — The property that repeating a request produces the same intended server-state effect as doing it once.
- **JSON (JavaScript Object Notation)** — A language-independent, standardized text format for structured data.
- **MCP (Model Context Protocol)** — A protocol standardizing how AI applications discover and interact with external tools, resources, and prompts.
- **MCP Client** — The component within an AI host/application that communicates with an MCP server using the MCP protocol.
- **MCP Host** — The AI application that wants to use external capabilities.
- **MCP Server** — An implementation exposing tools/resources/prompts via MCP, typically wrapping real integrations (APIs, databases, filesystems).
- **Path parameter** — A variable portion of a URL path identifying a specific resource instance (e.g., `42` in `/users/42`).
- **Prompts (MCP)** — Reusable prompt templates/workflows an MCP server can expose.
- **Query parameter** — Optional key-value data appended to a URL (e.g., `?notify=true`).
- **Resource** — The conceptual entity a REST API operates on (e.g., "user 42").
- **Resources (MCP)** — Contextual data an AI can read via an MCP server (files, DB info, docs).
- **REST (Representational State Transfer)** — An architectural style for networked APIs based on resources, standard HTTP semantics, and statelessness.
- **Route** — The URL pattern mapping a request to server-side handling logic.
- **RPC (Remote Procedure Call)** — An API style that models operations as function-like calls (e.g., `POST /getUser`) rather than resource-oriented URLs.
- **Statelessness** — Each request carries all information needed for the server to process it; no reliance on remembered prior-request context.
- **Status code** — A numeric HTTP response code indicating the outcome of a request (e.g., `200`, `404`).
- **Tools (MCP)** — Actions an AI can invoke through an MCP server (e.g., `create_issue()`).
- **WebSocket** — A protocol enabling persistent, two-way client/server communication, used for chat, live notifications, and similar real-time use cases.